GENMARK

Engineering Specification

Document type: Engineering Specification | **Status:** Pre-release | **Version:** 0.1-draft

Contents

1. System Overview & Security Objectives
2. Cryptographic Subsystem
3. Canonical Fingerprint Engine
4. Transform Layer - DCT-II / IDCT-III
5. Watermark Embedding Strategy - QIM
6. Payload Format & Error Correction
7. Threat Model
8. Algorithm-to-Threat Mapping
9. Known Limitations & Future Work

1 System Overview & Security Objectives

GenMark is a content-authenticity watermarking system. Its purpose is to bind a piece of video media to a verified cryptographic identity, such that any downstream consumer can determine whether the content originated from a trusted signer and whether the visual content has been substituted or significantly altered.

GenMark is explicitly not a file-integrity checker. It operates on visual content, not on bitstream bytes, and is therefore robust to the class of transformations that preserves visual appearance while changing the encoded representation - re-encoding, codec transcoding, container changes, and bitrate variation.

1.1 Security Objectives

Objective	Definition	Mechanism
Authenticity	Proof that the content was watermarked by the holder of a specific private key	Ed25519 signature over content-bound message
Integrity	Detection of significant visual substitution or alteration	Canonical fingerprint comparison at verification time
Non-repudiation	Signer cannot deny having watermarked a specific piece of content	Content-bound deterministic nonce baked into signed message
Robustness	Legitimate re-encodes of the same visual content must verify successfully	Perceptual fingerprint + QIM embedding with tuned Δ
Imperceptibility	Watermark must not degrade perceived visual quality below JND threshold	Mid-band DCT coefficient modification, Δ calibrated against SSIM

1.2 Pipeline Summary

The creation and verification pipelines are formally symmetric:

```
CREATION                                  VERIFICATION
──────────────────────────────────────    ──────────────────────────────────────
M  = original luma plane                  M' = candidate luma plane
F  = CanonicalFingerprint(M)              P  = ExtractWatermark(M')
n  = HKDF-SHA256(F, "genmark-v1")         (F_emb, n_emb, meta, S) = Deserialise(P)
S  = Ed25519_Sign(sk, SHA256(F|n|meta))   F' = CanonicalFingerprint(M')
P  = Serialise(F, n, meta, S)             n' = HKDF-SHA256(F', "genmark-v1")
M' = EmbedWatermark(M, P)                 msg'= SHA256(F'|n'|meta)
                                          Assert Ed25519_Verify(vk, msg', S)
                                          Assert F_emb == F'
```

Content-bound signature

> The deterministic nonce `n = HKDF-SHA256(F, "genmark-v1")` is derived from the fingerprint and included in the signed message. This means the signature is only valid for the specific visual content that produced `F`. See §7 for the transplant-attack analysis.

2 Cryptographic Subsystem

2.1 Library & Primitive Selection

All cryptographic operations are implemented via libsodium, statically linked. The library abstracts the following primitives behind two internal provider interfaces: SigningProvider and VerificationProvider.

Function	Algorithm	libsodium API	Key size / Output
Content hash	SHA-256	crypto_hash_sha256()	256-bit digest
HKDF nonce derivation	HKDF-SHA256	crypto_kdf_derive_from_key()	256-bit nonce
Signing	Ed25519	crypto_sign_detached()	64-byte signature
Verification	Ed25519	crypto_sign_verify_detached()	boolean

2.2 Ed25519 Signing

Ed25519 operates over the Edwards curve Curve25519 in its twisted-Edwards form. The signing equation is:

```
Key generation:
  (sk, vk) = Ed25519_KeyGen()     sk: 64 bytes (seed || public),  vk: 32 bytes

Signing:
  r  = SHA512(sk[0:32] || msg)    deterministic nonce (RFC 8032)
  R  = r * B                      B = base point of Ed25519
  S  = (r + SHA512(R || vk || msg) * s) mod l
  signature = R || S              64 bytes

Verification:
  (R, S) = signature
  k = SHA512(R || vk || msg)
  Assert: S * B == R + k * vk
```

The signed message `msg` is not the raw fingerprint. It is the compound value `SHA256(F || n || meta)`, where `F` is the canonical fingerprint and `n` is the content-bound HKDF nonce. This construction is described in §2.3.

2.3 Content-Bound Signature Construction

A naive implementation signs only the fingerprint F. This is vulnerable to a transplant attack (§7.3). The correct construction derives a deterministic nonce from the fingerprint and includes it in the signed plaintext:

```
// Nonce derivation (creation and verification must use identical parameters)
uint8_t nonce[32];
```

```
crypto_kdf_derive_from_key(
    nonce, sizeof(nonce),
    1,                          // subkey_id
    "genmark-v1\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0",  // context, padded to 8
bytes
    fingerprint                 // 32-byte master key input
);

// Signed message assembly
uint8_t msg[96];                // 32 (F) + 32 (nonce) + 32 (meta hash)
memcpy(msg,      fingerprint, 32);
memcpy(msg + 32, nonce,       32);
crypto_hash_sha256(msg + 64, metadata_bytes, metadata_len);

uint8_t sig[64];
crypto_sign_detached(sig, NULL, msg, sizeof(msg), sk);
```

Nonce derivation must be deterministic

If the nonce is random (not derived from F), two signatures of the same content with the same key will produce different payloads. The verifier cannot reconstruct the nonce from the content alone and verification will fail. The HKDF construction is mandatory.

3 Canonical Fingerprint Engine

The fingerprint engine converts a decoded video into a stable 32-byte digest that represents the visual content of the video rather than its bitstream. Two visually identical encodes of the same content must produce the same fingerprint. Two visually different videos must produce different fingerprints.

3.1 Design Constraints

Constraint	Implication
Codec-agnostic	Must not depend on any specific codec's internal representation
Decoder-agnostic	Same result regardless of whether FFmpeg, libav, or hardware decodes the frame
Deterministic across platforms	ARM and x86 builds must produce identical fingerprints for the same content
Robust to compression	Survives JPEG quantisation at QF ≥ 60, H.264 re-encode at CRF ≤ 28
Sensitive to substitution	Different visual content must produce different fingerprints

3.2 Frame Sampling - Scene-Based Selection

Index-based sampling (frame 0, 30, 60, ...) is fragile: a one-frame trim shifts all indices and produces a completely different fingerprint for visually identical content. The correct approach is **scene-based sampling** which anchors on content transitions rather than absolute positions.

```
// Scene-based frame selection
//
// 1. Decode every N-th frame (N = fps / 2, i.e. 2 samples per second)
// 2. Compute histogram of each frame's luma channel (256 bins)
// 3. Inter-frame histogram distance: sum of absolute bin differences
// 4. Identify the K largest scene transitions (default K = 8)
// 5. For each scene segment, select the frame at the temporal midpoint
//
// Rationale: midpoint frames are furthest from both scene boundaries
// and therefore least affected by encoder motion-blur at cuts.

float hist_distance(const uint32_t* h1, const uint32_t* h2, int bins) {
    float d = 0.f;
    for (int i = 0; i < bins; ++i)
        d += fabsf((float)h1[i] - (float)h2[i]);
    return d;
}
```

3.3 Per-Frame Perceptual Hash (DCT-pHash)

Raw pixel buffers cannot be hashed directly - minor decoder rounding differences produce wildly different SHA-256 outputs. A perceptual hash tolerates small pixel-level variations by

working in the DCT frequency domain where low-frequency energy dominates visual appearance.

The per-frame hash algorithm:

1. Resize the frame to 32×32 pixels using Lanczos resampling with a fixed kernel. The kernel and parameters must be identical across all platforms - stb_image_resize is bundled for this reason.
2. Convert to grayscale using the fixed formula: `Y = 0.2126·R + 0.7152·G + 0.0722·B` (ITU-R BT.709). The formula must be hardcoded - do not rely on FFmpeg's automatic colorspace selection, which depends on container metadata.
3. Apply a 2D DCT-II to the 32×32 luma block, producing a 32×32 coefficient matrix.
4. Extract the top-left 8×8 sub-block (64 low-frequency coefficients, excluding DC).
5. Compute the mean μ of those 64 coefficients.
6. For each coefficient `c_i`: `bit_i = (c_i > μ) ? 1 : 0`. Output: 64-bit hash per frame.

Why the DCT domain is robust

JPEG and H.264 both quantise DCT coefficients. Low-frequency coefficients are quantised with small step sizes (high precision), so they survive re-encoding almost unchanged. The 8×8 top-left extraction isolates these stable components. The mean-threshold comparison is invariant to global amplitude scaling, making the hash robust to mild brightness/contrast changes as well.

3.4 Hamming Threshold

Matching is done by Hamming distance on the per-frame 64-bit hashes. Empirical values:

Scenario	Typical Hamming distance	Decision
Same content, different codec	0–3 bits	Match (threshold ≤ 8)
Same content, heavy JPEG (QF 60)	2–6 bits	Match
Same content, mild brightness change	1–4 bits	Match
Different scene, same video	20–40 bits	No match
Completely different video	28–36 bits	No match

The aggregated fingerprint is `SHA256(phash_0 || phash_1 || ... || phash_{K-1})` where frames are ordered by scene position (ascending temporal midpoint), not by decode order. This ordering is stable under container re-mux.

4 Transform Layer - DCT-II / IDCT-III

The transform layer converts spatial pixel data to frequency-domain coefficients and back. The current implementation is DCT-II over 8×8 luma blocks, matching the JPEG baseline transform. This section specifies the exact equations, implementation decisions, and the interface contract that future transforms must satisfy.

4.1 Mathematical Definition

The 2D DCT-II (forward transform) of an 8×8 block f(x,y) is:

```
                7   7
C(u,v) = α(u)·α(v)·(1/4) · Σ   Σ   f(x,y) · cos[(2x+1)·u·π/16] ·
cos[(2y+1)·v·π/16]
                x=0 y=0

where:  α(k) = 1/√2  if k = 0
        α(k) = 1      otherwise
```

The inverse transform (DCT-III / IDCT) reconstructs f(x,y) from C(u,v):

```
                7   7
f(x,y) = (1/4)·Σ   Σ   α(u)·α(v) · C(u,v) · cos[(2x+1)·u·π/16] ·
cos[(2y+1)·v·π/16]
                u=0 v=0
```

The (1/4)·α(u)·α(v) normalisation makes this the orthonormal form: for any 8×8 block, `IDCT(DCT(f)) = f` holds exactly in infinite-precision arithmetic. In double-precision floating point the roundtrip error is sub-LSB (< 0.5) for all pixel values in [0, 255], allowing round-clamp reconstruction without visible loss.

4.2 Cosine Lookup Table

The cosine values are identical in both directions: `cos[(2x+1)·u·π/16]` appears in both the forward and inverse equations with the same arguments. A single compile-time table serves both passes:

```
// Built at compile time - consteval, lives in .rodata
// kCos[u][x] = cos((2x+1) * u * π / 16)

consteval CosLUT build_cos_lut() {
    CosLUT lut{};
    for (int u = 0; u < 8; ++u)
        for (int x = 0; x < 8; ++x)
            lut[u][x] = std::cos((2.0*x + 1.0) * u * std::numbers::pi / 16.0);
    return lut;
}
constexpr CosLUT kCos = build_cos_lut();
```

Using consteval eliminates the runtime initialisation race condition present in the original implementation and places the table in read-only memory, making it safe to use from any number of concurrent threads without synchronisation.

4.3 Padding Strategy

Video frames decoded by FFmpeg carry two distinct dimension concepts: the coded dimensions (padded to macroblock alignment, typically multiples of 16 or 32) and the visible/crop dimensions. The watermark operates on the visible dimensions as specified in FrameGeometry.

When the visible width or height is not a multiple of 8, the right column and bottom row of blocks will overlap the edge. The implementation extends these blocks by boundary replication (clamp-to-edge) before the forward pass:

```
// Boundary replication during padded buffer fill
for (int y = 0; y < padded_height; ++y) {
    const int sy = std::min(y, geo.height - 1);   // clamp row
    for (int x = 0; x < padded_width; ++x) {
        const int sx = std::min(x, geo.width - 1); // clamp col
        padded[y * padded_width + x] = luma[sy * geo.stride + sx];
    }
}
```

On the inverse pass, only visible pixels (px < geo.width, py < geo.height) are written back. Padding columns and rows are discarded. This is identical to the behaviour of libjpeg and standard hardware JPEG encoders, ensuring that a GenMark-watermarked frame re-encoded by JPEG does not introduce edge artefacts beyond the expected quantisation noise.

4.4 Stride Handling

FFmpeg allocates luma buffers with stride ≥ width for SIMD alignment. The transform reads pixels as `src[y * geo.stride + x]` rather than `src[y * geo.width + x]`. Writing back uses the same stride. This is not optional: reading with the wrong stride shifts every row after the first, producing entirely wrong DCT coefficients for all non-zero block rows.

4.5 ITransform Interface Contract

Any future transform (DWT, FFT) must satisfy the following contract to be a valid ITransform implementation:

Method	Precondition	Postcondition
forward(media, geo)	media.size() ≥ geo.stride × geo.height; geo.width > 0, geo.height > 0, geo.stride ≥ geo.width	Returns Ok; internal coefficient buffer populated; coeff() is valid
inverse(media, geo)	forward() has been called successfully on this instance; same geometry	Returns Ok; visible pixels in media updated; padding bytes unchanged
coeff(p) - read	forward() returned Ok; p within block_cols/block_rows/freq_u/freq_v	Returns the current coefficient value

Method	Precondition	Postcondition
	bounds	
coeff(p) - write	Same as read	Updated value is used by the subsequent inverse() call
forward() before inverse()	inverse() called on fresh instance	Returns Status::InvalidState

5 Watermark Embedding Strategy - QIM

Watermark embedding is the process of encoding the serialised payload P into the frequency-domain coefficients produced by the transform layer. The chosen scheme is Quantisation Index Modulation (QIM), a blind watermarking scheme that does not require the original media for extraction.

5.1 QIM Algorithm

QIM encodes one bit by quantising a coefficient to one of two interleaved lattices:

```
Lattice Λ_0 = { ..., -2Δ,  0,  2Δ,  4Δ, ... }   (even multiples of Δ)
Lattice Λ_1 = { ...,  -Δ,  Δ,  3Δ,  5Δ, ... }   (odd  multiples of Δ)

Embedding:
    q = round(c / Δ)          // nearest lattice point index
    if (bit == 0): c' = q_even * Δ    where q_even = q rounded to nearest even
    if (bit == 1): c' = q_odd  * Δ    where q_odd  = q rounded to nearest odd

    Compact form:
    c' = (round(c/Δ - bit/2) + bit/2) * Δ      // single expression

Extraction (blind - no original needed):
    bit = round(c' / Δ) mod 2
```

The parameter Δ (quantisation step size) is the primary robustness control. It must satisfy `Δ ≥ 2 · q(u,v)` where `q(u,v)` is the JPEG quantisation step for the chosen coefficient position at the target quality factor. See §5.3 for calibration.

5.2 Coefficient Selection

Fixed coefficient positions are a security weakness - an attacker who knows the positions can zero them without solving the signature. Selection must be pseudo-random, seeded by the fingerprint:

```
// Coefficient selection seeded by fingerprint
void select_coefficients(
    const uint8_t fingerprint[32],
    int           n_needed,
    std::vector<CoeffPos>& out
) {
    // PRNG seeded from fingerprint - ChaCha20 or AES-CTR
    CSPRNG prng(fingerprint);

    // Pool: zigzag indices 15..41 (mid-band, avoids DC and high-freq)
    std::vector<CoeffPos> pool = enumerate_midband_coefficients();

    // Fisher-Yates shuffle
    for (int i = pool.size() - 1; i > 0; --i) {
```

```
        int j = prng.next_uint32() % (i + 1);
        std::swap(pool[i], pool[j]);
    }
    out.assign(pool.begin(), pool.begin() + n_needed);
}
```

Selecting coefficients from the fingerprint means an adversary must know the private key AND control the content fingerprint to predict which coefficients carry the watermark. Any change to the visual content that shifts the fingerprint also shifts the coefficient selection, causing extraction to fail.

5.3 Δ Calibration Against JPEG Quantisation

The JPEG standard quantisation table for luminance at quality factor Q assigns each coefficient (u,v) a step size q(u,v). For the watermark to survive one JPEG round-trip at quality Q, the embedding Δ must dominate the quantisation step:

```
JPEG luma quantisation table (quality 50 baseline, standard):
  [ 16  11  10  16  24  40  51  61 ]
  [ 12  12  14  19  26  58  60  55 ]
  [ 14  13  16  24  40  57  69  56 ]   ← rows correspond to DCT rows u=0..7
  [ 14  17  22  29  51  87  80  62 ]
  [ 18  22  37  56  68 109 103  77 ]
  [ 24  35  55  64  81 104 113  92 ]
  [ 49  64  78  87 103 121 120 101 ]
  [ 72  92  95  98 112 100 103  99 ]

At quality Q:  q_Q(u,v) = clamp(floor(q_50(u,v) * (100-Q) / 50 + 0.5), 1, 255)

Mid-band range (zigzag 15..27) at Q=70:  q_Q ≈ 5..18
Required Δ_min = 2 * max(q_Q in band) ≈ 36
Recommended Δ for QF ≥ 60: 32  (empirically validated)
```

Δ must be validated empirically using the test protocol in §9.1. The above derivation gives a floor; the actual optimal Δ is the smallest value that achieves BER < 0.01 before ECC at the target quality range.

5.4 Capacity

The payload P (see §6) must fit within the available coefficient capacity. Available capacity per frame:

```
Mid-band pool size:    zigzag indices 15..41 = 27 coefficients per 8×8 block
Blocks in a 1920×1080 frame (after padding): 240 × 136 = 32,640 blocks
Total available:       32,640 × 27 = ~880,000 coefficient slots

GenMark payload size (§6.2):  ~256 bits (ECC-encoded) → ~512 bits with
BCH(63,7)
Capacity utilisation:  512 / 880,000 < 0.06%
```

```
Capacity is not a concern at any practical video resolution.
For audio watermarking (future), FFT-domain capacity will be re-evaluated.
```

6 Payload Format & Error Correction

6.1 Wire Format

```
Offset  Size    Field
──────  ──────  ──────────────────────────────────────────────────────
0       1       version          u8, currently 0x01
1       32      fingerprint      SHA-256 canonical fingerprint (F)
33      32      nonce            HKDF-derived content-bound nonce (n)
65      4       metadata_len     u32 little-endian, max 256
69      N       metadata         JSON or binary, schema versioned
69+N    64      signature        Ed25519 signature of SHA256(F||n||meta)
──────  ──────  ──────────────────────────────────────────────────────
Total (no metadata): 133 bytes
Total (max metadata 256 bytes): 389 bytes

After ECC encoding the bit count approximately doubles (BCH overhead).
All fields are big-endian unless noted.
```

6.2 BCH Error Correction

Single-coefficient bit errors from JPEG quantisation are inevitable even with a well-tuned Δ. An ECC layer is mandatory. GenMark uses BCH(63,7) which corrects up to 10 bit errors per 63-bit codeword.

```
BCH(63,7) parameters:
  n = 63    codeword length in bits
  k = 7     information bits per codeword
  t = 10    maximum correctable bit errors per codeword

Overhead ratio: 63/7 = 9×  (high, but payload is small - see §5.4)

Encoding procedure:
  1. Split payload into 7-bit chunks
  2. Encode each chunk to a 63-bit BCH codeword
  3. Interleave codewords across the coefficient pool
     (interleaving spreads burst errors from JPEG block boundaries)
  4. Embed each bit into a selected coefficient using QIM

Extraction procedure:
  1. Extract each bit from its coefficient using QIM decode
  2. De-interleave to reassemble BCH codewords
  3. Apply BCH decoder - corrects up to 10 flips per 63-bit block
  4. Concatenate decoded 7-bit chunks to recover payload bytes
```

Interleaving is critical for JPEG robustness. JPEG applies per-block quantisation, so a high-quantisation block may flip many consecutive bits from the same codeword. Interleaving maps consecutive payload bits to coefficients in different blocks, turning burst errors into isolated errors that BCH can correct.

7 Threat Model

This section defines the adversary model, enumerates concrete attack vectors, and states the design's claimed protection against each.

GenMark's goal is **content authentication**, not confidentiality. The system does not attempt to hide the existence of the watermark or the payload contents. The cryptographic guarantee is that a valid payload can only be produced by the holder of the private signing key for content whose visual fingerprint matches the embedded fingerprint.

7.1 Adversary Model

Adversary capability	In scope?
Knows the full embedding algorithm and wire format	Yes - assume full white-box knowledge
Can extract the payload from any watermarked video	Yes - extraction is public
Can re-encode, transcode, compress, or scale a watermarked video	Yes
Can embed arbitrary bytes into the watermark region	Yes - with knowledge of the scheme
Possesses the public verification key	Yes
Possesses the private signing key	No - key security is assumed
Can produce fingerprint collisions at will	No - SHA-256 preimage resistance assumed
Can break Ed25519 signatures without the private key	No - DLP hardness assumed

7.2 Attack: Watermark Removal

Objective: Strip the watermark so the video cannot be verified, without destroying visual quality.

Attack methods:

- Aggressive JPEG/H.264 re-encoding to destroy mid-band coefficients.
- Spatial filtering (Gaussian blur, median filter) to suppress high-frequency energy.
- Targeted zeroing of DCT mid-band coefficients (requires knowing which coefficients carry the watermark).

Mitigation: Δ is calibrated to survive re-encoding at the target quality floor (QF ≥ 60). Below this floor, visible quality degradation occurs before the watermark is destroyed - an attacker faces a quality/removal tradeoff. Pseudo-random coefficient selection (§5.2) prevents targeted zeroing without knowing the private key or the fingerprint.

Residual risk: A determined attacker with unlimited re-encodes can drive BER above the BCH correction threshold. The system makes this expensive, not impossible.

7.3 Attack: Payload Transplant

Objective: Extract a valid payload from an authenticated video V_A and embed it into a different video V_F, causing the verifier to accept V_F as authentic.

```
Naive transplant attempt:
  1. Extract P_A = (F_A, n_A, meta_A, S_A) from V_A
  2. Embed P_A into V_F
  3. Submit V_F to verifier

Verifier check 1:  Ed25519_Verify(vk, SHA256(F_A||n_A||meta_A), S_A)  → PASS
                   (S_A is a valid signature - attacker did not modify it)
Verifier check 2:  F_A == CanonicalFingerprint(V_F)                   → FAIL
                   (V_F has different visual content, different fingerprint)
```

First-order protection: The fingerprint comparison catches all naive transplants. An attacker cannot pass check 2 unless they produce V_F with the same fingerprint as V_A.

Second-order transplant: fingerprint collision attack

An attacker who can construct V_F such that CanonicalFingerprint(V_F) == F_A would pass both checks. The content-bound signature (§2.3) closes this: even with a matching fingerprint, the nonce `n_A = HKDF(F_A, "genmark-v1")` equals `n_F = HKDF(F_F, "genmark-v1")` only if F_A == F_F. If the attacker produces a genuine fingerprint collision, both protections fail simultaneously - but this requires breaking SHA-256 preimage resistance, which is outside the adversary model.

7.4 Attack: Payload Replay

Objective: Reuse a previously valid payload P_A from an old version of a video to authenticate a newer, altered version.

Mitigation: The fingerprint F embedded in P_A is the fingerprint of the original content. If the content has changed enough to matter (scene replacement, significant edit), CanonicalFingerprint(V_new) will differ from F_A and check 2 fails. The content-bound nonce also changes, so a payload from a previous signing of different content cannot be replayed even if the fingerprint change happens to be small.

An attacker cannot simply update F in the payload - changing F invalidates S, because the signature covers SHA256(F||n||meta) and changing F changes both the signed message and the nonce.

7.5 Attack: Watermark Overwrite

Objective: Replace the embedded payload with a forged one claiming different metadata or a different signer identity.

Mitigation: The attacker can embed any bit pattern they like. However, they cannot produce a valid Ed25519 signature without the private key. The verifier will reject any payload where the

signature does not verify against the trusted public key. There is no partial forgery - Ed25519 produces 64-byte signatures with no malleable structure an attacker can exploit without the key.

7.6 Attack: Adversarial Re-compression

Objective: Apply targeted compression that destroys the watermark while preserving visual quality, by specifically degrading the mid-band coefficients that carry the payload.

Mitigation: The coefficient selection is pseudo-random seeded by the fingerprint (§5.2). An attacker does not know which specific coefficients carry the payload without knowing the fingerprint in advance - and knowing the fingerprint requires decoding the existing payload, which they can extract from the watermark. However, knowing the fingerprint in advance does not help: the attacker still cannot produce a valid signature for altered content. Adversarial re-compression can destroy the watermark (making the video unverifiable) but cannot produce a forged valid watermark.

8 Algorithm-to-Threat Mapping

The following table maps each attack from §7 to the specific algorithm or system property that mitigates it, and identifies any residual risk.

Attack	Primary mitigation	Relevant section	Residual risk
Watermark removal by re-encoding	QIM with Δ ≥ 2×max(q_JPEG) + BCH(63,7) error correction + interleaving	§5.3, §6.2	Below QF 60 an attacker can destroy the watermark. Quality floor is a design parameter, not a security guarantee.
Watermark removal by spatial filtering	Mid-band DCT coefficients are above the frequency range attenuated by typical blur kernels (σ < 1.5px)	§5.2	Strong blur (σ > 3px) may attenuate mid-band. Visual degradation at that level is usually unacceptable.
Targeted coefficient zeroing	Pseudo-random selection seeded by SHA-256(fingerprint) - attacker cannot predict positions without fingerprint	§5.2	Attacker can extract fingerprint from the payload and predict positions. Can destroy watermark, not forge it.
Naive payload transplant	Fingerprint comparison at verification: CanonicalFingerprint(V_candidate) must match F_embedded	§3, §2.3	None if fingerprint is stable and collision-free.
Fingerprint collision + transplant	Content-bound HKDF nonce: n = HKDF(F, "genmark-v1") binds signature to specific F	§2.3	Requires SHA-256 collision - outside adversary model.
Payload replay	F and n both change when content changes; changing either invalidates S	§2.3, §6.1	Imperceptible content edits that do not change fingerprint are undetectable by design.
Payload overwrite / forgery	Ed25519 signature - no known attack short of private key theft	§2.1, §2.2	Key management is out of scope. Key theft defeats all cryptographic protections.
Adversarial targeted re-compression	Coefficient positions unknown without fingerprint; destroying positions only removes watermark, not forges it	§5.2	Same as watermark removal - quality tradeoff still applies.

Out-of-scope: key management

Every cryptographic guarantee in GenMark depends on the private signing key remaining secret. Key theft, key leakage through side channels, or compromise of the signing environment invalidates all protections. Key management (HSM usage, key rotation, revocation) is a deployment concern outside this specification.

9 Known Limitations & Future Work

9.1 Validation Test Protocol

Before release, the following test matrix must be run. All passing thresholds are required:

Test	Input	Expected result	Pass threshold
Codec roundtrip	H.264 → H.265 → H.264 re-encode	Verifies successfully	100% of test corpus
Bitrate stress	Re-encode at CRF 28, 32, 36	Verifies at CRF ≤ 32	> 99% at CRF 28
JPEG frame stress	Extract frames, JPEG at QF 60, 70, 80	All verify	100% at QF ≥ 70, 95% at QF 60
Container change	MP4 → MKV → WebM	Verifies successfully	100%
Transplant	Embed payload from V_A into V_B	Verification fails	100% detection
Trim 5s from start	Trim, re-verify	Verification fails	100% detection
False positive rate	10,000 random video pairs	No false verification	0 false positives
Roundtrip error	Forward + inverse, no modification	Max pixel error ≤ 1	100%

9.2 Fingerprint Sensitivity to Colour Grading

The current pHash mean-threshold is not invariant to global contrast changes greater than ±25%. A normalised rank-based variant can address this without changing the interface:

```
// Rank-based variant (invariant to global amplitude scaling)
std::array<uint8_t, 64> ranked;
std::iota(ranked.begin(), ranked.end(), 0);
std::sort(ranked.begin(), ranked.end(),
    [&](int a, int b){ return coeffs[a] < coeffs[b]; });

// bit_i = 1 if coefficient i is in the upper half by rank
for (int i = 0; i < 64; ++i)
    hash_bit[ranked[i]] = (i >= 32) ? 1 : 0;
```

This changes the fingerprint value for all existing signed content - it cannot be introduced as a minor update. It requires a version bump in the payload format and a re-signing of all existing content.

9.3 Future Transform Support (DWT)

The ITransform interface is designed to accommodate DWT (Discrete Wavelet Transform) as used in JPEG2000 and some H.265 profiles. DWT operates on the full frame rather than 8×8

blocks, offering potentially better robustness against block-boundary artifacts. The key interface changes needed:

- freq_u() and freq_v() return the full frame dimensions divided by 2 (for one decomposition level) rather than 8.
- CoeffPos.u and CoeffPos.v address wavelet subbands (LL, LH, HL, HH) rather than DCT frequency indices.
- The watermark strategy must be updated to interpret CoeffPos in the DWT context - but ITransform itself requires no change.

9.4 Audio Watermarking

The ITransform comment in transform.hpp notes FFT as a future candidate for audio tagging. The same ITransform interface can serve audio with the following interpretation mapping:

FrameGeometry field	Audio interpretation
width	FFT window size (e.g. 1024 samples)
height	Number of windows per chunk
stride	Window hop size (samples between windows)

The watermark strategy, payload codec, and cryptographic subsystem are all media-agnostic and require no changes for audio support.